CPE 010 · DATA STRUCTURES AND ALGORITHMS · FINAL PROJECT

Game Store

A Steam-style game platform for the terminal, written in C++17

Mendoza · Pangilinan · Pasion · Reyes · Torres | Technological Institute of the Philippines

Speaker 1 · Mendoza · Introduction

WHAT THE PROGRAM DOES

One store, three kinds of users

Players

- Accounts with hashed passwords
- Store table, search, tags, sorting
- Cart, wallet, wishlist, reviews
- Downloads and a bundle optimizer

Developers

- Separate developer account
- Developer Studio: publish a game with title, tags and price
- Edit only their own games
- Owners and rating per game

Social and discover

- Friends and live chat between terminals
- Friend network and suggestions
- Featured carousel, genre tree, history
- Algorithm lab and lesson showcase

HOW IT IS BUILT

Small modules, one lesson each

23
files: 22 headers and main.cpp

333
functions and methods, all documented

14
of 14 lessons applied

5
unit-test programs

Layer	Files	Job
Data	game.h, account.h, review.h, cart.h, friends.h	Records kept in linked lists and trees, saved to text files
Lessons	stack.h, queue.h, ring.h, algorithms.h, genre.h, graph.h, dp.h	One data structure or algorithm per file, as templates
Screens	store.h, screens.h, tui.h, main.cpp	Menus, tables and the program entry point

HOA 2.1 · 3 · 4 · 5

Lists, stacks and queues

Structure	File	Where the player sees it
Singly linked list	game.h, account.h, cart.h	Catalog, accounts, cart, wishlist, friends
Circular doubly linked list	ring.h	Featured carousel: next after the last game shows the first
Stack (array and linked)	stack.h	Back and Forward, cart undo
Queue (linked)	queue.h	Download queue
Circular queue (array)	queue.h	Recently viewed: the oldest game drops out

Every structure has its own cleanup, and the classes follow the rule of three: destructor, copy constructor and copy assignment.

STACKS AND QUEUES AT WORK

Four features, four structures

Back / Forward (two stacks)

Opening a game pushes the current page on the back stack and clears the forward stack. Back pops one stack and pushes the other.

Cart undo (linked stack)

Each add or remove is recorded. Undo pops the last action and reverses it.

Download queue (linked queue)

Games are installed first in, first out by one worker thread.

Recently viewed (circular queue)

Five slots. The sixth game overwrites the oldest one, and the indexes wrap to 0 after the last place.

HOA 6 · 7 · 8

Searching and sorting

Sort	Average	Stable	Idea
Bubble	O(n²)	yes	Swap neighbours, stop after a pass with no swap
Selection	O(n²)	no	Put the smallest item first, at most n-1 swaps
Insertion	O(n²)	yes	Slide each item left into place
Shell	O(n^1.5)	no	Insertion sort with a shrinking gap
Merge	O(n log n)	yes	Split, sort halves, merge with a temp array
Quick	O(n log n)	no	Middle pivot, recurse on the smaller side

Searching: linear search O(n), binary search O(log n) on a sorted array, and a binary search tree for titles and usernames. The store page itself is sorted by insertion sort.

HOA 11 · 14 · ALGORITHM LAB

Measured, not just claimed

10,000 random numbers	Time (ms)	Comparisons
Bubble	216.2	49,994,054
Selection	141.6	49,995,000
Insertion	12.3	25,084,451
Shell	1.18	268,371
Merge	0.99	120,353
Quick	0.82	184,693

Already sorted input

Bubble sort: 0.007 ms (early exit). Selection sort: 141 ms, the same as for random data.

Searching 1,000,000 items

Linear: 1,000,000 comparisons. Binary: 21.

Measured with -O2 on the build machine, times vary by computer. The comparison counts do not.

HOA 9 · TREES

Binary search trees and a genre tree

Binary search trees

Games are found by title and accounts by username. Each step goes left or right, so a lookup is O(log n) on a balanced tree. Capital letters do not matter, and the lists keep the order for the store table.

General tree of genres

19 genres, height 3: action > shooter, platformer, fighting; rpg > roguelike; and more. Each node has a first child and a next sibling, so a node can have any number of children.

Traversal	First six genres
Preorder	all games, action, shooter, platformer, fighting, adventure
Postorder	shooter, platformer, fighting, action, puzzle, story
Level order	all games, action, adventure, rpg, strategy, simulation

HOA 10 · 12

Graphs and dynamic programming

Friend network (graph)

- Users are vertices, friendships are edges
- Adjacency list for traversal, matrix for O(1) checks
- BFS with a queue: friends at 1, 2 and 3 steps and the shortest chain
- DFS with a stack: the size of your friend group
- Friends of friends: people you may know

Dynamic programming

- Fibonacci of 25: 242,785 calls with plain recursion, 49 with memoization
- Knapsack: the wishlist games with the best total rating that fit the budget
- Edit distance: “Did you mean …?” when a search finds nothing

HOA 13 · MULTITHREADING

Two threads and several terminals

Chat thread

Checks the chat file every 300 ms and prints new messages at once. A mutex guards the unread counters, a second mutex guards the console.

Download worker

Waits on a condition variable for the next game in the queue. Progress is an atomic value that the screen reads.

Several terminals

Lock the file, reload, change, save through a temporary file, unlock. No terminal overwrites another.

Race test: 6 terminals at once

6 terminals each added $10 five times to one wallet: exactly $300.00. All 7 accounts registered at the same time were saved, none lost.

QUALITY

How we checked the project

Documentation

333 functions. Each one has a comment above it that says what it does and where it is used. A function index with line numbers sits at the top of every file. docs/DOCUMENTATION.md is generated by our docgen tool.

Tests

5 unit test programs in tests/. We also ran the code with AddressSanitizer, UBSan and ThreadSanitizer, with no errors reported. The documentation parser was cross-checked against libclang.

DEMO AND SUMMARY

Run it, then questions

Live demo

1. Run main.cpp
2. Create an account and log in
3. Discover, Run all lessons (or run with --showcase)
4. Open the Algorithm lab

Summary

- 14 of 14 lessons used in one working program
- 23 files, 333 documented functions, 5 tests
- Safe with several terminals open

Thank you. Any questions?